

LightWork AI - Data Engineer Take-Home Submission

This document summarizes my proposed data architecture for LightWork AI's platform, designed to support operational automation, real-time processing, and agentic AI.

What this submission covers

- PMS ingestion for legacy and modern API providers
- Real-time processing for messages, calls, and transcripts
- AI context assembly, action orchestration, and benchmarking
- Scale strategy with measurable intervention thresholds

Review focus

- Reliability: idempotency, replay, and auditability
- Security: tenant isolation and PII-aware processing
- Practicality: startup-ready stack and phased rollout
- Performance: latency, quality, and cost as first-class metrics

Repository Link: <https://github.com/egenc/lightwork-data-engineer-takehome>

1) Problem and Design Goals

LightWork needs one reliable platform that can ingest data from many PMS systems, process live communication events, and provide safe, high-quality context to AI agents.

Primary goals

- Ingest legacy and modern PMS data without data loss
- Support both batch and real-time workflows
- Protect tenant PII and provide compliance-ready controls
- Scale gradually with clear cost and latency thresholds

Guiding principles

- Use proven tools first; add complexity only when required
- Build for replay, idempotency, and observability from day one
- Keep architecture understandable for both engineers and operators

2) Architecture for PMS Ingestion and Storage

Ingestion approach

- **Connector workers (Python):** one adapter per PMS provider
- **Legacy APIs:** polling + checkpoints + retries + backoff
- **Modern APIs:** webhooks + signature checks + periodic reconciliation
- **Transport:** Kafka topics for durable, replayable event flow

Medallion data model

Bronze (Raw)

Immutable source payloads in object storage and raw event topics. This layer is used for auditing and replay.

Silver (Conformed)

Normalized records with dedupe and policy checks. Canonical tenancy entities live in PostgreSQL; searchable text is indexed in OpenSearch.

Gold (Serving)

AI-ready context views combine structured and unstructured data for agent decisions, dashboards, and analytics.

Data stores and purpose

- **PostgreSQL:** source of truth for tenants, tenancies, and actions
- **OpenSearch:** low-latency transcript/message retrieval
- **Object storage:** low-cost raw archives (audio, transcripts, payloads)
- **Redis (optional):** cache, idempotency windows, short-lived coordination state

3) Real-Time Processing and AI Orchestration

Live data flow

- Messages/calls/webhooks enter a realtime ingest API
- Events are published to Kafka realtime topics
- Stream processors enrich events with tenant and conversation context
- AI workers classify intent, urgency, and next-best action
- Action engine emits recommendations or triggers approved workflows

Airflow + Kubernetes responsibilities

Airflow

- Schedules PMS sync and reconciliation jobs
- Runs controlled backfills
- Provides operational DAG visibility

Kubernetes

- Runs connector, consumer, and AI worker deployments
- Autoscaling based on lag and latency
- Isolates noisy workloads by namespace or pool

Performance targets

- P95 ingest-to-classification latency under 3 seconds
- P95 ingest-to-recommendation latency under 8 seconds
- End-to-end observability across ingest, transform, and AI actions

4) Compliance, Benchmarking, and Scale Plan

Compliance and governance (current offering)

- Encryption in transit and at rest
- Tenant-scoped access patterns and auditability
- PII-aware prompt handling with masking rules
- Retention policy by data class (events, transcripts, raw media)

AI quality and safety benchmarking

- **Offline:** precision/recall/F1, ranking quality, safety violation rate
- **Online:** shadow mode, A/B tests, resolution and escalation impact
- **Operations:** latency, model cost per resolved case, drift indicators

Scale interventions (when to act)

- Add consumers when queue lag is sustained beyond threshold
- Add Redis when repeated hot reads impact user-facing latency
- Partition high-volume tables and move cold data to cheaper storage
- Adopt dedicated vector infrastructure when semantic retrieval latency grows

Implementation posture: Startup-practical by default, production-safe by design, and explicitly measurable through latency, quality, and cost KPIs.